

DATA/CS 172: Python for Data Analysis

Jupyter notebooks

Prof. Travis Mandel

11/6/2024



UNIVERSITY
of HAWAI'I®

HILO

Reminders

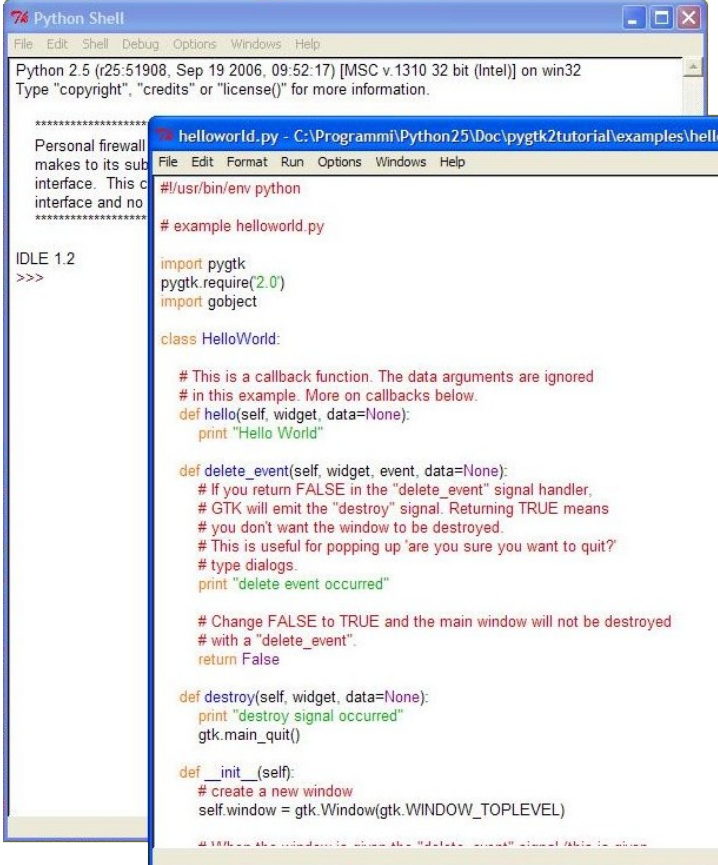
- Stay on top of labs
- Make progress on Program 2
- Office hours today 12:30-1:30pm
- Teaching Demo 5-6pm (including discussion of numpy arrays!)
 - Will send Zoom link on Discord
- Test next Wednesday
 - Review Friday
 - (Monday is a holiday)

Today

- Jupyter notebooks

So far, we've been editing our code using IDEs

- For example, IDLE
- Makes some things nice
 - Easy to run code, simple, error indications
- But showing graphs, etc. is a bit of a pain
- Some of the data science community has gravitated towards an alternate solution
 - One that makes Python code easier to create, share, and run
 - More accessible to non-programmers
- Jupyter!
 - The key element of Jupyter is called the Jupyter notebook



```
Python Shell
File Edit Shell Debug Options Windows Help
Python 2.5 (r25:51908, Sep 19 2006, 09:52:17) [MSC v.1310 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.

*****
Personal firewall
makes to its sub
interface. This c
interface and no
*****

IDLE 1.2
>>>

helloworld.py - C:\Programmi\Python25\Doc\pygtk2tutorial\examples\hellow
File Edit Format Run Options Windows Help

#!/usr/bin/env python

# example helloworld.py

import pygtk
pygtk.require(2.0)
import gobject

class HelloWorld:

    # This is a callback function. The data arguments are ignored
    # in this example. More on callbacks below.
    def hello(self, widget, data=None):
        print "Hello World"

    def delete_event(self, widget, event, data=None):
        # If you return FALSE in the "delete_event" signal handler,
        # GTK will emit the "destroy" signal. Returning TRUE means
        # you don't want the window to be destroyed.
        # This is useful for popping up 'are you sure you want to quit?'
        # type dialogs.
        print "delete event occurred"

        # Change FALSE to TRUE and the main window will not be destroyed
        # with a "delete_event".
        return False

    def destroy(self, widget, data=None):
        print "destroy signal occurred"
        gtk.main_quit()

    def __init__(self):
        # create a new window
        self.window = gtk.Window(gtk.WINDOW_TOPLEVEL)
```

Jupyter: Many ways to access

- Way 1: Install locally (not recommended)
 - pip install jupyter
 - Open command line, type “jupyter notebook”
 - If it doesn't work, run similar to how you run pip
 - Opens up in a browser
 - Click New → Notebook → Python 3
- Way 2: JupyterLite
 - **The easiest way - recommended!**
 - NEW – started being an option in 2022
 - Runs entirely in the browser – nothing to install!
 - Go to <https://jupyter.org/try-jupyter/lab/>
 - Click on Python (pyodide) under “Notebook”
- Way 3: Use Google Colaboratory
 - Go to Google Drive
 - Click New → More → Google Colaboratory
 - For UH, need to enable external apps on your account
 - For personal account, need to install the app in Drive
 - Notebook will run
- In either case, double click title to change filename



IPython

IP[y]:

- Stands for interactive Python
- Write some Python code
 - Press shift-enter (or click  Run or  or ) to run it.
- But wait, we saw interactive mode in Python the second day of class!
 - This seems super similar
 - Has some fun additional features...

IPython feature: Tab completion

- Type part of a variable name, then hit tab – it will autocomplete
 - Or type variable name then . and <tab>, to see what you can do with it (append to a list for instance)
 - For this last one, must have run code where it was initialized first



IPython Feature: Introspection

- Hit ?<variable name> to see information about the variable
 - What type it is
 - The values
 - The length
 - Etc.

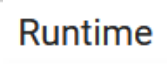
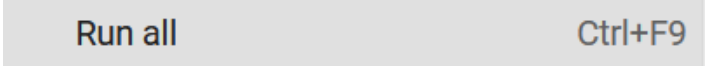


Downloading/Saving notebooks

- JupyterLite: Right Click File → Download
 - Jupyter: File → Save and Checkpoint
 - Colaboratory: File → Download ipynb
- Creates a .ipynb file In the directory you ran the command from
 - This is a self-contained file
 - Also contains the output from the latest run – helpful if you want someone else to see what output you got

Jupyter feature: Running snippets of code

- What if I want to change something above?
 - Need to re-run that piece of code
 - Jupyter “remembers” what has been done in each previously-run block

Can press  **Run** or  or  to run just single snippet,
or  or  **Runtime** **Tools** **Help** [All changes saved](#) to run all blocks
 **Run all** **Ctrl+F9**

- Jupyter allows you to run code blocks in any order
 - Warning: Can mess stuff up if run pieces of code out of order
 - Can be annoying/confusing
- Can use up and down arrows to re-order blocks.

Jupyter Feature: Graphs

- Doing graphs in matplotlib is a bit of a pain...
- Have to do `plt.show()` which halts execution
 - Or `plt.savefig()` which saves a figure, then have to open it up etc.
- Jupyter allows us to combine graphs with code!
 - Can show graphs on screen right next to the section of code that generates it
 - No need to do `plt.show()` – Jupyter will auto-display results from `plt.plot()` etc.
 - Used to need “`%matplotlib inline`” for this, but no longer needed
 - Note: May take a bit longer to run (especially the first time) - * shown while processing
 - `plt.show()` will remove the annoying “Line2D” thing though
 - But does not stop the code from continuing!
 - One of the biggest reasons Jupyter is popular for data science



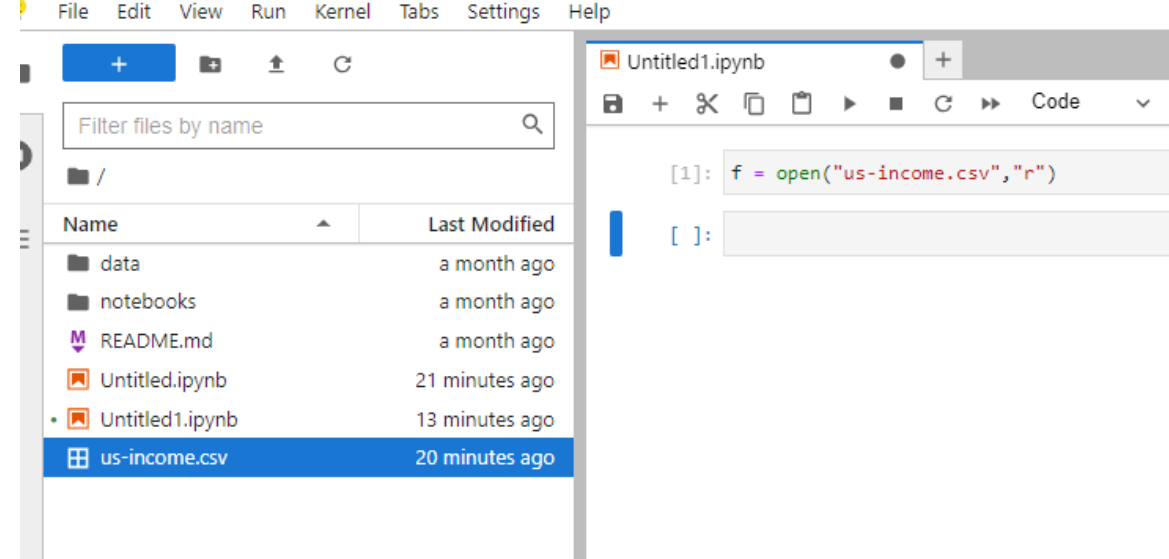
Jupyter feature: Interspersing Headings, text, and equations



- Select “Markdown”,
“raw markdown,”
or “Text” then:
 - # Top level Heading
 - ## Second level heading
- Text
 - Select raw markdown to write freeform text
 - Often a replacement for block comments – describes what we are doing & why
 - Run to finalize, double-click to edit
 - Can include LaTeX equations
 - x^2
 - $\sqrt{x}$
 - $\frac{x_0}{x_1}$
 - Don’t expect you to learn Latex in this course...
 - But at some point you really should.. Makes formatting long scientific documents much easier
- Can even include HTML tags and links (`<a href=""></a>`) etc.

Loading data

- JupyterLite:
 - Drag and drop to data folder (left hand side)
 - Then can open as normal
 - May have trouble with certain Firefox installs (works fine in Chrome)
 - Uses browser storage to save files (NOT good for larger files – there is a limit of 50MB)
- Jupyter notebook:
 - Will look for data in the same folder that you ran the “jupyter notebook” command from
- Colaboratory:
 - This is more difficult, unfortunately
 - Similar to JupyterLite, but data deleted frequently
 - This is due to the fact that Colaboratory is run on Google’s servers
 - Google Drive is another option, but requires other code + permissions



When it's all put together

- It's effectively an interactive report!

Simple spectral analysis

An illustration of the [Discrete Fourier Transform](#) using windowing, to reveal the frequency content of a sound signal.

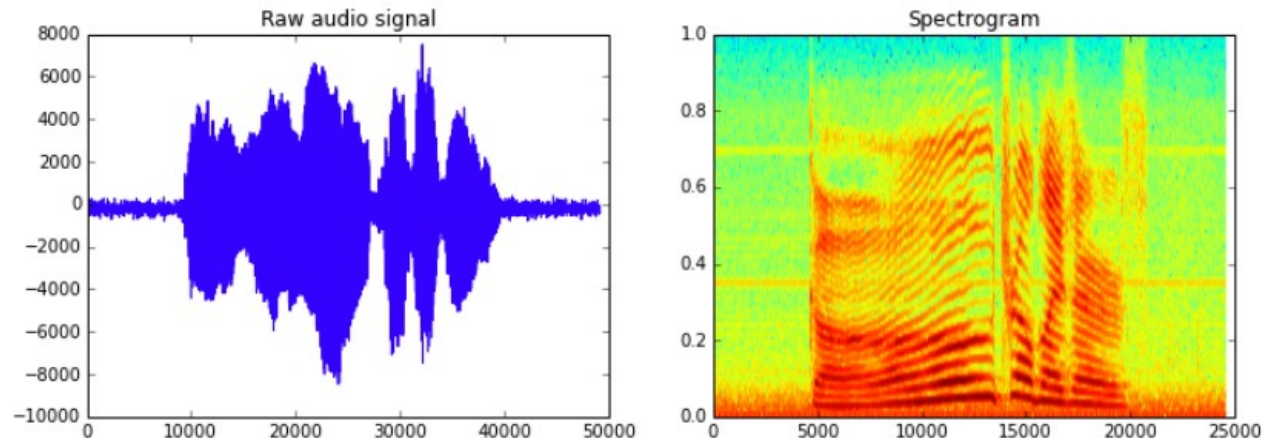
$$X_k = \sum_{n=0}^{N-1} x_n e^{-\frac{2\pi i}{N} kn} \quad k = 0, \dots, N-1$$

We begin by loading a datafile using SciPy's audio file support:

```
In [1]: from scipy.io import wavfile
rate, x = wavfile.read('test_mono.wav')
```

And we can easily view its spectral structure using matplotlib's builtin spectrogram routine:

```
In [2]: %matplotlib inline
from matplotlib import pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(x); ax1.set_title('Raw audio signal')
ax2.spectrogram(x); ax2.set_title('Spectrogram');
```



Lab out

- Creating a simple Jupyter notebook
 - Can use any method but I think JupyterLite is the easiest!
- Next up: Review and Test 2